

提交说明文档

一、团队信息

项目	内容
团队名称	吃饱饱
A 榜排名	赛道二 3
A 榜分数	84.3
联系人/联系方式	李祥；微信：17561841398；电话：17561841398

二、项目概述

（一）核心任务

对带噪点云进行去噪。输入是噪声（A 榜为 Laplace 噪声）污染的三维点云，输出是同形状的干净点云。评测指标为 CD（Chamfer Distance）和 P2S（Point to Surface）的改善比例，等权合成 Final 分数。官方提供训练 mesh 和测试 noisy 点云。

（二）解决思路

（1）数据层面：

将官方 OBJ mesh 按三角形面积采样为 50,000 点的 clean 点云，做单位球归一化；

训练时不使用固定噪声版本，而是在线生成 Laplace 噪声，让模型见到更丰富的噪声分布；

从训练 shape 中确定性切 200 个作为验证，在训练过程中选择 best model

（2）推理层面：

整云太大无法直接送模型，采用 patch 策略：FPS 选 97 个中心 → 每个中心取 2048 点 KNN patch → 各 patch 减去中心坐标后送模型 → 模型输出加回中心 → poly 加权拼回整云；

niters=2 跑两轮去噪，第一轮粗去噪、第二轮细化；

patch 中心权重高、边缘权重低，消除 patch 边界接缝

（三）模型选型

采用 三阶段级联的可逆流去噪网络（HeavyDenoiseFlow），三个 DenoiseFlow 串联：



单个 DenoiseFlow 的内部流程：

1. KNN 图构建：在 patch 内对每个点取 32 近邻
2. 局部几何特征提取：10 层 EdgeConv(类 DGCNN)，用 point + neighbor difference 建模局部几何
3. 噪声通道预测：NoiseEdgeConv 根据邻域预测 32 个附加噪声通道，与原始 3 维坐标拼接为 35 维
4. 可逆流变换：10 个 FlowAssembly（每个含 2 个 IMonotoneBlock + 2 个 ActNorm）将 35 维映射到潜空间
5. 噪声通道截断：将潜空间最后 16 个通道清零（cut_channel=16），假设噪声因素被分离到这些通道
6. 逆变换恢复：通过可逆流的逆变换将截断后的潜变量映射回 3 维坐标空间

可逆性保障： InducedNormLinearJT 对线性层做 Lipschitz 约束（coeff=0.98），通过幂迭代更新谱范数，保证固定点求逆收敛。推理时复用 checkpoint 中保存的 u/v/scale 状态，不重新迭代。

（四）算法创新点

相较于 Baseline，我们的模型核心创新在于引入了可逆流（Invertible Flow）结构。主要创新点如下：

1. 潜空间显式噪声分离

- Baseline：特征提取后直接通过 MLP 回归坐标，属于黑盒式拟合，网络需要隐式学习噪声与几何的分离，学习难度大且不稳定。

- Ours：通过 NoiseEdgeConv 预测附加的噪声通道，与坐标拼接后送入 Flow 模型。在潜空间中，我们显式地截断（置零）代表噪声的 cut_channel 个

维度。这在物理上模拟了“信息过滤器”，强制网络将去噪转化为潜空间维度裁剪的确定性操作，具有更强的可解释性和鲁棒性。

2. 三阶段渐进式去噪

- **Baseline**：单阶段端到端预测，网络需要一次性完成从噪声到干净的映射，容易陷入局部最优。

- **Ours**：采用级联的 HeavyDenoiseFlow（Stage A/B/C），并为每个阶段定义了 渐进式目标：

- Stage 1：仅去除大部分噪声（保留少量残差）
- Stage 2：进一步细化去噪
- Stage 3：输出最终干净结果

这种由粗到精的课程学习策略大幅降低了单个网络的优化难度，显著提升了高噪点下的恢复精度。

3. 稳定的可逆流架构：为了保证 Flow 的可逆性，我们引入了 InducedNormLinear（诱导范数线性层）和基于固定点迭代的 IMonotoneBlock。通过谱范数约束（Lipschitz Constant < 1），严格保证了非线性变换的可逆性，解决了生成模型在点云处理中训练不稳定的问题。

4. 多维度质量约束: 设计了 EMD + Chamfer + Repulsion 组合损失: EMD：采用 Auction 算法求解一一对应，有效防止点云整体漂移； Repulsion Loss：惩罚距离过近的点，防止生成点云聚集现象，保持点云的拓扑分布均匀；这些机制弥补了 Baseline 仅依赖 L2 损失的局限性，使生成结果在视觉和几何上更自然。

三、代码结构说明

（一）目录说明

路径	内容
code/main.py	统一命令行入口，封装全部 7 个子命令
code/prepare_data.py	OBJ 网格转标准化 NPY 点云、离线构建训练/验证集
code/data.py	数据集类、点云归一化、训练 Patch 成对采样、在线加噪增强
code/model.py	自研三阶段 HeavyDenoiseFlow 可逆流去噪网络
code/emd.py	CUDA 即时编译 Auction EMD 距离计算模块
code/train.py	模型训练、续训、梯度优化、周期验证、最

路径	内容
	优模型保存
code/validate.py	Held-out 验证集批量推理、CD/P2S/Final 指标计算、最优模型筛选
code/denoise.py	测试集分 Patch 推理、多轮去噪、加权拼接、输出校验
code/datalist/	训练、验证、测试样本 ID 清单文件
code/scripts/env.sh	环境变量一键配置脚本
code/scripts/prepare_data.sh	数据预处理封装脚本
code/scripts/train_single_gpu.sh	单卡训练封装脚本
code/scripts/infer_single_gpu.sh	单卡推理+校验+打包一体化脚本
code/scripts/full_pipeline_three_commands.sh	据-训练-推理完整流水线脚本
requirements.txt	依赖清单
提交说明文档.pdf	官方提交文档

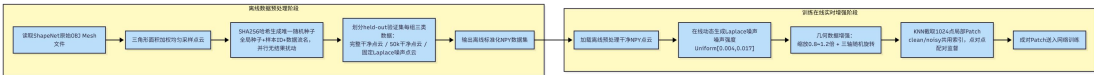
（二）核心文件功能说明

文件名称	核心功能
main.py	基于 argparse 实现统一 CLI 入口，提供 7 个子命令：prepare-data、verify-test、train、validate、infer、validate-predictions、package-predictions，统一调度全流程功能
prepare_data.py	读取 ShapeNet 原始 OBJ 网格，按三角形面积泊松采样点云；包围盒中心化 + 单位球缩放归一化；基于哈希派生确定性种子，离线生成 15633 组训练点云、200 组 held-out 验证集（含 clean、clean_50k、带固定 Laplace 噪声 noisy），输出 manifest 审计文件记录样本哈希、随机种子、数据类型
data.py	实现 PairedPatchDataset 训练数据集类；完成点云归一化、在线动态生成 Laplace 噪声；随机缩放、旋转数据增强；KNN 切分 1024 点局部 Patch，保证 noisy/clean Patch 索引一一配对，输入网络前中心化 Patch 消除绝对坐标干扰
model.py	定义 HeavyDenoiseFlow 三阶段级联模型，单阶段 DenoiseFlow 包含：PreConv+10 层 EdgeConv 局部特征提取、NoiseEdgeConv 噪声特征通道预测、10 组可逆 FlowAssembly 潜空间映射；谱范数约束线性层 Lipschitz 常数 ≤ 0.98 ；潜空间截断后 16 维噪声通道实现去噪；三阶段输出共同参与损失监督
emd.py	Jittor 即时编译 CUDA Auction EMD 算法，固定 1024 点计算、 $\epsilon=0.005$ 、迭代 50 次；大幅提升点集匹配损失计算速度，支撑训练阶段多阶段 EMD 损失
train.py	模型从零随机初始化训练，支持断点续训；完整训练循环：加载干净点云→在线加噪增强→切分 Patch→三阶段网络前向传播；总损失 = EMD 加权 + CD 加权 + 排斥损失；Adam 优化器 + Plateau 学习率衰减，梯度裁剪稳定训练；每 5 个 epoch

文件名称	核心功能
	执行完整验证，仅当 200 组验证样本全部完成且 Final 分数严格提升时保存最优模型权重与训练状态
validate.py	加载训练 Checkpoint 在 200 项 held-out 数据集批量推理；计算双向 CD、点到网格 P2S 距离，合成 Final 综合得分；校验全部样本推理完成后更新最优模型存档与分数汇总 json
denoise.py	测试集推理核心模块：点云单位球归一化；FPS 采样 Patch 中心、KNN 取 2048 点 Patch；两轮迭代去噪、分批次前向降低显存占用；多项式加权融合多 Patch 预测；反归一化还原原始坐标，严格保证输出点数与输入一致；内置输出校验逻辑，过滤 NaN/Inf 异常值
scripts/env.sh	自动配置 PYTHONPATH、JITTOR 运行环境、nvcc/g++ 编译器路径，运行所有 shell 脚本前自动加载，无需手动配置环境变量
scripts/prepare_data.sh	封装数据预处理全流程，一键读取原始 OBJ 数据集输出标准化 NPY 训练 / 验证集
scripts/train_single_gpu.sh	单卡训练脚本，支持传入断点续训状态文件，自动输出 Checkpoint、验证指标明细
scripts/infer_single_gpu.sh	一体化推理脚本，顺序执行：输入校验→批量去噪推理→输出格式校验→自动打包 A 榜提交 result.zip
scripts/full_pipeline_three_commands.sh	端到端完整流水线，一条命令完成数据准备、模型训练、测试推理打包全流程

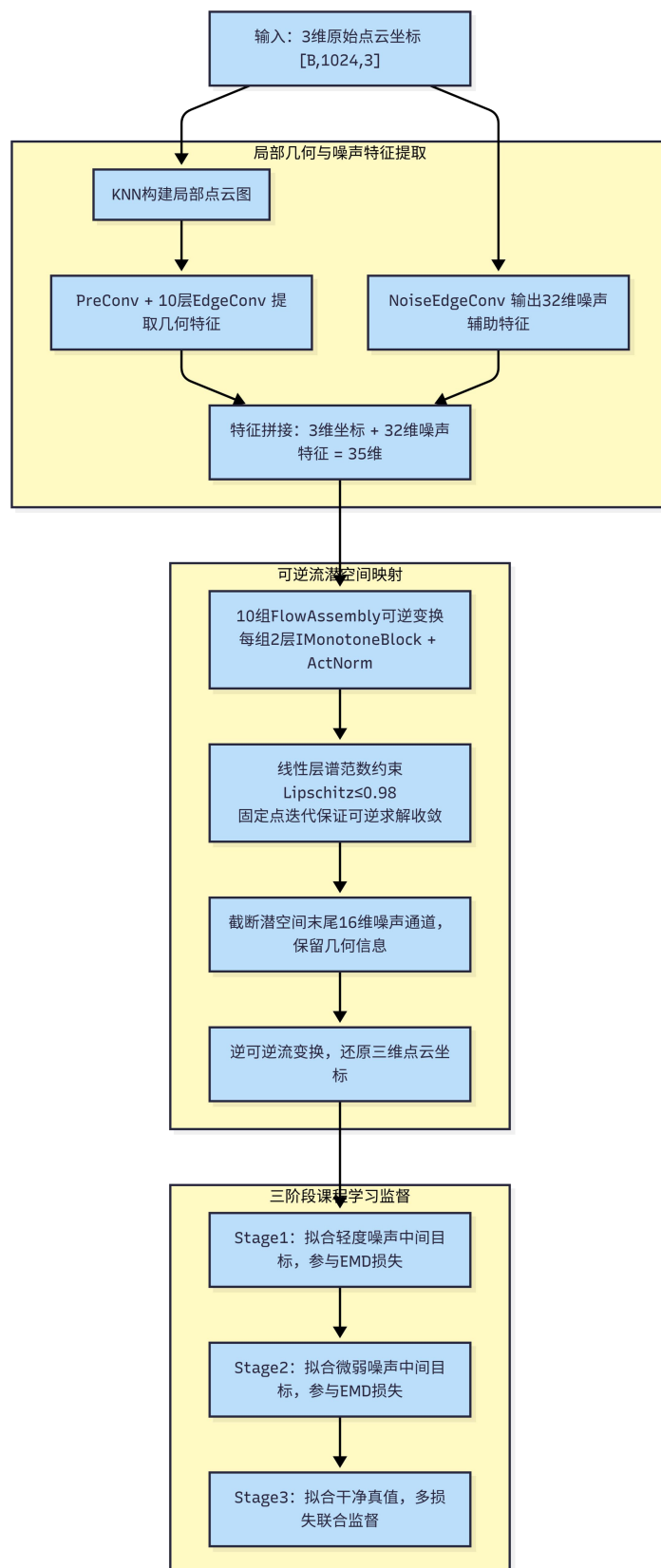
（三）关键模块逻辑说明

（1）数据预处理模块（prepare_data.py + data.py）



（2）网络模型模块（model.py）

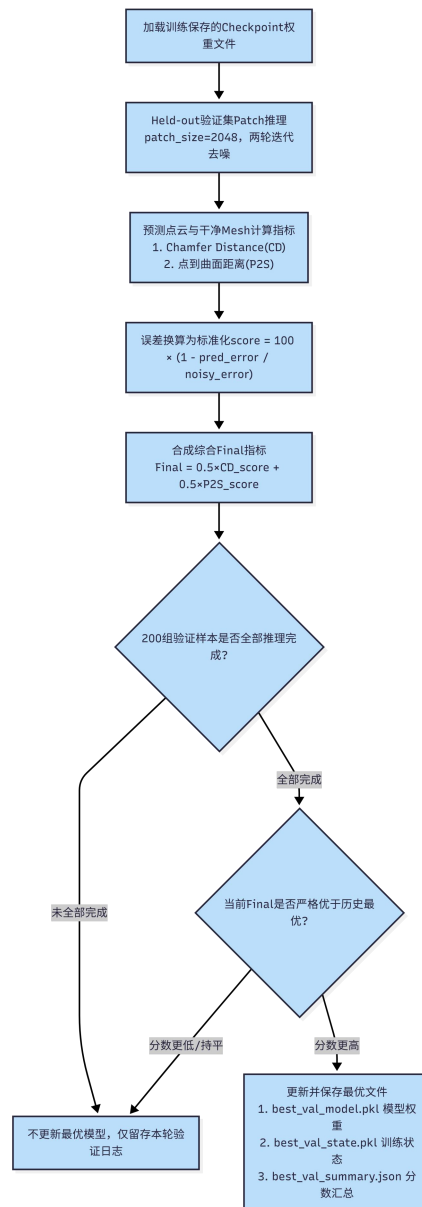
单阶段 DenoiseFlow 流程：



(3) 训练模块 (train.py)

训练无预训练权重，完全随机初始化；Batch 输入为成对 1024 点局部 Patch；损失权重固定：EMD 权重 0.00625、CD 权重 40.0、排斥损失权重 40.0；排斥损失限制 $k=8$ 近邻距离不小于 0.05，防止点聚集；优化器采用 Adam，初始学习率 $2e-4$ ，Plateau 策略衰减，学习率下限 $1e-6$ ；梯度裁剪最大范数 0.001 避免梯度爆炸；每 5 个完整 epoch 执行全量 held-out 验证，仅全部 200 样本推理完成且 Final 分数提升，才覆盖最优模型文件。

(4) 验证模块 (validate.py)



(5) 推理模块 (denoise.py)



四、环境配置步骤

(一) 系统基础要求

操作系统：Ubuntu 22.04

CUDA 版本：12.4

Python 版本：3.9

硬件要求：NVIDIA 显卡

(二) 环境一键安装流程

```
python -m pip install -r requirements.txt
```

验证 CUDA 与 Jittor 运行环境是否正常：

```
python -c 'import jittor as jt; jt.flags.use_cuda=1; print("Jittor 版本：", jt.__version__)'
```

五、运行步骤

前置说明

数据集目录规范：

1. 训练原始 OBJ 数据集：

/path/to/dataset_train/shapenet/<synset>/<model>/models/model_normalized.obj

2. 测试噪声 NPY 数据集：

/path/to/dataset_test_noisy/shapenet/<synset>/<model>/noisy.npy

```
bash code/scripts/full_pipeline_three_commands.sh \  
  /path/to/dataset_train \  
  /path/to/dataset_test_noisy \  
  /path/to/full_output_root \  
  0
```

（一）方案一：全流程自动化

参数释义：

1. /path/to/dataset_train: 训练集根目录
2. /path/to/dataset_test_noisy: 测试集根目录
3. /path/to/full_output_root: 结果文件保存根目录

4.0: 使用 GPU 0 进行训练与推理

运行完成后产出：

最优模型权重：

full_output_root/train_rotation_none/checkpoints/best_val_model.pkl

预测结果：full_output_root/test_best_val/predictions/shapenet/*/*/denoised.npy

提交结果：full_output_root/test_best_val/result.zip

（二）方案二：分阶段分步运行

（1）数据预处理

```
bash code/scripts/prepare_data.sh \  
  /path/to/dataset_train \  
  /path/to/prepare_data_root \  
  rotation_none_epoch98
```

输出内容：

prepared_data_root/train/*.npy: 15633 组 50k 干净训练点云

prepared_data_root/heldout/shapenet/*/*/: 200 组验证集（clean、clean_50k、noisy）

prepared_data_root/manifest.json: 全量样本审计记录（种子、哈希、尺寸）

(2) 模型训练

从头训练:

```
bash code/scripts/train_single_gpu.sh \  
  /path/to/prepared_data_root/train \  
  /path/to/prepared_data_root/heldout \  
  /path/to/dataset_train \  
  /path/to/train_output_root \  
  0
```

参数说明:

TRAIN_DATA_ROOT: 训练集 NPY 目录

VAL_ROOT: 验证集目录

OBJ_MESH_ROOT: 训练集 OBJ 目录

RUN_ROOT: 输出目录 (保存权重、验证日志)

GPU_INDEX: 显卡编号

断点续训:

```
RESUME_STATE=/path/to/train_output_root/checkpoints/latest_state.pkl \  
bash code/scripts/train_single_gpu.sh \  
  /path/to/prepared_data_root/train \  
  /path/to/prepared_data_root/heldout \  
  /path/to/dataset_train \  
  /path/to/resume_train_output_root 0
```

训练产出文件:

checkpoints/best_val_model.pkl: 最优模型权重

checkpoints/latest_state.pkl: 完整训练断点 (模型、优化器、epoch、最优分数)

best_val_summary.json: 最优验证 CD/P2S/Final 分数

validation/epoch_XXXX/: 每 5 轮验证样本指标明细

(3) 模型推理

推理前指定最优模型权重 CKPT, 执行一体化推理脚本:

```
CKPT=/path/to/train_output_root/checkpoints/best_val_model.pkl \  
bash code/scripts/infer_single_gpu.sh \  
  /path/to/dataset_test_noisy \  
  /path/to/infer_output_root \  
  0
```

脚本自动串行执行 4 个子流程：

verify-test: 校验测试集 200 个样本目录、文件完整性

infer: 分 Patch 批量去噪推理，生成 denoised.npy

validate-predictions: 校验输出点数量、float32 格式、无 NaN/Inf

package-predictions: 打包 result.zip 提交文件

推理产出：

去噪点云：

infer_output_root/predictions/shapenet/<synset>/<model>/denoised.npy

提交结果：infer_output_root/result.zip

六、核心超参数说明

（一）训练

参数	固定取值	说明
max-epochs	100	最大训练轮次
batch-size	16	单次迭代输入 Patch 数量
train-patch-size	1024	训练局部 Patch 点数
lr	2e-4	初始学习率
min-lr	1e-6	学习率衰减下限
plateau-patience	2	验证分数不提升时衰减等待轮数
plateau-factor	0.5	学习率衰减系数
emd-weight	0.00625	EMD 损失权重
cd-weight	40.0	CD 损失权重
repulsion-weight	40.0	repulsion 损失权重
repulsion-radius	0.05	repulsion 损失最小近邻距离阈值
validate-every	5	每 4 个 epoch 执行一次完整验证
seed	2023	全局固定随机种子
grad-clip	0.001	梯度裁剪最大范数
noise-min / noise-max	0.004/0.017	Laplace 噪声强度区间

（二）推理

参数	固定取值	说明
patch-size	2048	推理单块 Patch 点数（大于训练 Patch 扩大感受野）
seed-k	4	FPS 采样密度系数，控制 Patch 总数约 97 个
niters	2	点云完整去噪迭代轮次，两轮细化提升效果
patch-batch	12	单次前向传播 Patch 批次，控制显存占用
stitch	poly	Patch 融合加权模式
stitch-power	1.0	权重衰减指数

参数	固定取值	说明
rotation	identity	推理阶段不使用旋转增强